WEB UI AND CONTENT MANAGEMENT · PROJECT REPORT

Plannr

The college timetable, attendance and scheduling web application for SICSR

Course	Web UI and Content Management, BCA, Semester VII, Division A
Assessment	Assessment #3: Project (15 marks)
Institute	Symbiosis Institute of Computer Studies and Research (SICSR)
Group members	Paras Deshpande (PRN 23030124026), Sonal Mohrir, Tanisha Nath
Faculty	Shashank T. Gaikwad
Date	11 October 2026
Live application	https://plannr.paraswtf.com
Project files	https://github.com/paraswtf/plannr https://github.com/paraswtf/plannr/tree/web-ui-compliance/web-ui-compliance

Index

1 Introduction 2
2 Problem Statement 2
3 Objectives 2
4 Scope of the Project 3
5 Technologies Used 3
6 Implementation 5
7 Screenshots 7
8 Testing 9
9 Individual Contribution 9
10 Conclusion 9
11 Future Scope 10
12 References 10

1 Introduction

Plannr is a web application that gives every student, teacher and coordinator of SICSR a personal view of the college timetable, and builds attendance, change requests and scheduling on top of it. It is live at https://plannr.paraswtf.com and its source code is public at https://github.com/paraswtf/plannr.

The project began as a design-thinking study of how timetables are used at the college. We interviewed and surveyed the people involved, found that the existing system serves the whole college with one page and no personal view, and built Plannr as the response. This report describes the problem, the objectives, the technologies chosen, the implementation, the testing and the contribution of each member.

2 Problem Statement

The college timetable is published at time-table.sicsr.ac.in, an installation of MRBS, a room-booking system written in PHP on MySQL (a LAMP application). It is one page for the whole college, designed for a desktop and served over plain HTTP; its day view is a single 141 KB HTML document listing every room (measured on 11 October 2026). Coordinators schedule each lecture by hand through the “Add Entry” booking form, and information about changes is spread over the timetable site, Gmail, the class WhatsApp group and Moodle.

Our research covered 15 students (four open questions), 11 students (survey), 10 faculty members from SICSR and SIG, and 4 coordinators (interviews).

Finding	Result
Survey of 11 students: the timetable does not tell them when a class changes	73%
Too many steps to find their own classes	82%
Find the timetable cluttered	82%
Cannot see their week at a glance	82%
Say it is slow, buggy or crashes	64%
Interviews: students who raised changes not reaching them	8 of 15
Students who use WhatsApp to learn about changes	87%
Coordinators who announce every change by hand (mail, then WhatsApp)	3 of 4
Coordinators who type up the day’s report of classes held by hand	2 of 4

Faculty set up attendance by hand after every lecture: “Every lecture I lose five minutes just setting up attendance.”

3 Objectives

1. Show each person only their own timetable, in day, week and month views, usable on a phone.
2. Create an attendance register for every class automatically from the timetable.
3. Let faculty request an extra lecture, a room change or a cancellation, and let coordinators approve from a single queue without double-booking.
4. Detect clashes of room, teacher, group and capacity, and suggest the next free slot.
5. Generate a clash-free weekly timetable from demand, with a review and approval workflow.
6. Mirror classes into the user’s Google Calendar, and import people and rooms in bulk from CSV with a preview.
7. Apply the syllabus concepts (Ajax, DOM, server-side scripting with a database) in a working system.

4 Scope of the Project

Scope of the product

Plannr as a product covers the timetable, attendance, class scheduling and tracking, and tests and assignments for a college.

What this submission implements

- Personal timetable (day, week, month; swipe between days; opens on the next class).
- Google Calendar sync (opt-in); cancelled classes are retitled, not deleted.
- Attendance registers from the timetable: present, absent, late, excused; students see attended, held and remaining per course against the threshold.
- Requests and approvals with clash display; clash detection; room booking with recurring bookings.
- Timetable planner with review workflow (draft, in review, approved, applied).
- Recurrence stored as RFC 5545 rules; bulk CSV import with a diff preview.
- College Google sign-in, permissions as a bitfield, REST API with OpenAPI and API keys, Siri shortcuts, installable web app.

Out of scope

File uploads and course material are outside the scope of Plannr. Features of the wider product that are not built in this submission (notifications, tests and assignments among them) are listed honestly in Section 11.

5 Technologies Used

Technology	Role in Plannr
Next.js 15, React 19	App Router and server components; React renders and updates the DOM
TypeScript (strict)	One language and type system from database to browser
tRPC	Typed RPC over HTTP; JSON requests from the browser without page reloads, batched
Prisma ORM, PostgreSQL (Supabase, Mumbai)	Database access with pooled connections
Auth.js v5	Google sign-in restricted to the sicsr.ac.in domain, checked on the server
Tailwind CSS v4	Styling and the design system
Vercel	Serverless hosting, functions pinned to Mumbai (bom1)
REST API at /api/v1	OpenAPI 3.1 document, scoped API keys
Google Calendar API	Opt-in sync, drained by a cron job
Vitest	Unit and integration tests

Mapping to the syllabus

Syllabus concept	How it is applied
Ajax	Every screen loads and saves data with asynchronous JSON calls (tRPC and the REST API).
DOM	React renders and updates the DOM, with direct DOM APIs where needed (swipe gestures, measuring).
Database connectivity, server-side scripting	Prisma and PostgreSQL in Node.js server code, the role PHP and MySQL play in LAMP.
LAMP	Plannr imports and reconciles the college's own LAMP timetable (an MRBS export) with scripts that preview every class added or removed and verify against the source.

The syllabus allows additional technologies where required; we chose the following stack for the reasons below.

Why not LAMP, MySQL or WordPress

- **An application, not content pages.** Plannr has scheduling logic (clash checks, a planner that generates a week, recurrence, locks); a CMS such as WordPress or Drupal would need all of it as custom plugins anyway.
- **One language end to end.** TypeScript from the database schema to the browser through Prisma and tRPC catches mistakes at build time; PHP with jQuery would split the code in two.
- **PostgreSQL over MySQL.** Transaction-scoped advisory locks (so concurrent approvals cannot double-book), strict constraints, transactional schema changes, and a managed host (Supabase) with pooling for serverless.
- **Serverless over a LAMP server.** Nothing to patch or keep running, scales to zero on the free tier, HTTPS by default, a preview deployment for every change, and functions placed next to the database in Mumbai.
- **The limits of the existing LAMP timetable:** one 141 KB page for the whole college, desktop only, plain HTTP, no per-person view, no notifications.

5.1 The legacy version (web-ui-compliance branch)

For the course's prescribed stack we also built a legacy version: a small, working re-creation of the timetable viewer on LAMP (Linux, Apache, MySQL, PHP 8 with PDO) with jQuery, jQuery UI and Ajax, rendering directly into the DOM. It is the folder `web-ui-compliance/` on the `web-ui-compliance` branch: https://github.com/paraswtf/plannr/tree/web-ui-compliance/web-ui-compliance. The database (MySQL 8, InnoDB) holds rooms, student groups, faculty, courses, classes and class changes (moved, cancelled or extra, with reason and time), seeded with a realistic BCA week. PHP endpoints return JSON: `api/groups.php`, `api/day.php` (a group's classes for a date, with any change applied), `api/week.php` (week grid), `api/faculty.php` (teacher search: where they are now and when next free) and `api/changes.php` (recent changes; POST a move, cancellation or extra class, validated on the server with a clash check on room, teacher and group that returns 409 with the clashing class). The page `index.php` uses `$.getJSON` and `$.ajax`, a jQuery UI datepicker, dialog and autocomplete, and polls the changes feed every 30 seconds; a coordinator form (behind a demo `?role=coordinator` switch, not real authentication) posts changes by Ajax. `.htaccess` sets the Apache rules and hides `config.php`.

6 Implementation

Architecture

The browser runs a Next.js application. Pages are rendered with React server components; interactive screens call typed tRPC procedures over HTTP, and external clients use the REST API. Server code on Vercel (Mumbai) reads and writes PostgreSQL on Supabase (Mumbai) through Prisma. The schema has 41 Prisma models, from Organization, User and Role through Event, EventOccurrence, EventChangeRequest, AttendanceSession and AttendanceRecord to SchedulePlan, CalendarMirror and ApiKey. Figure 1 shows the entity-relationship diagram.

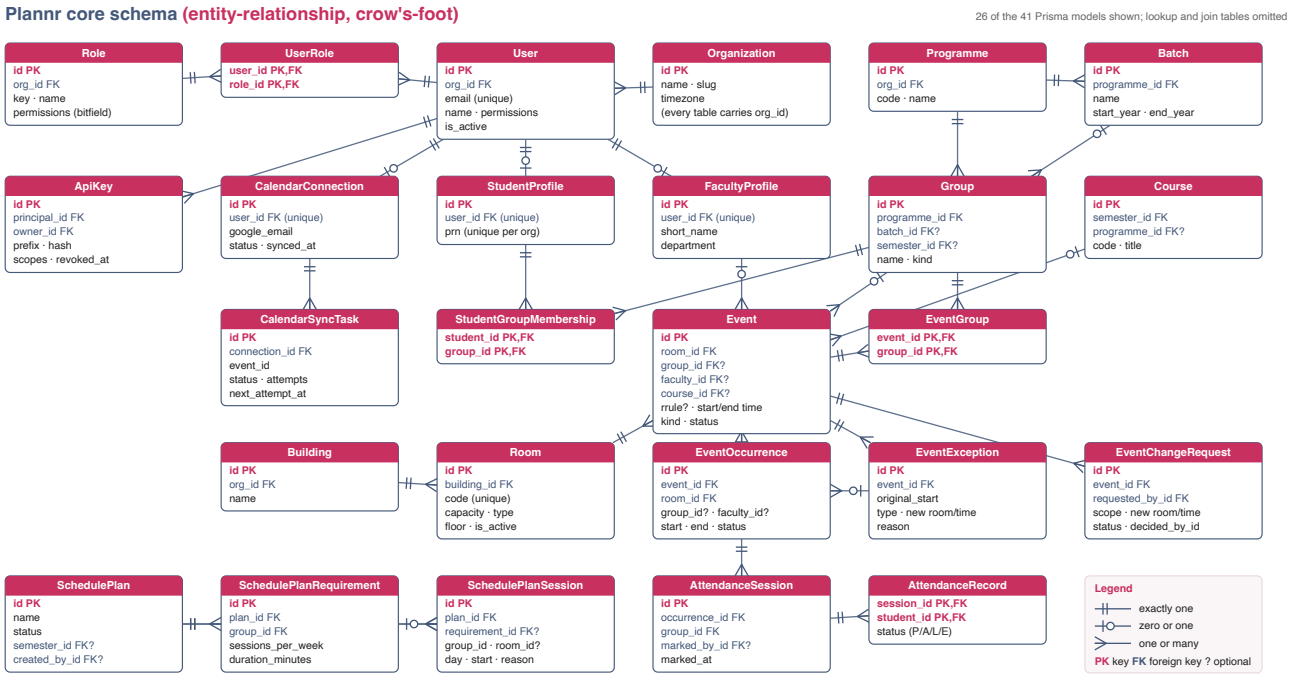


Figure 1: Entity-relationship diagram of Plannr's database.

Data flow

Figure 2 gives the level-0 data-flow diagram: the system between its external actors. Figure 3 breaks it into the level-1 processes and data stores.

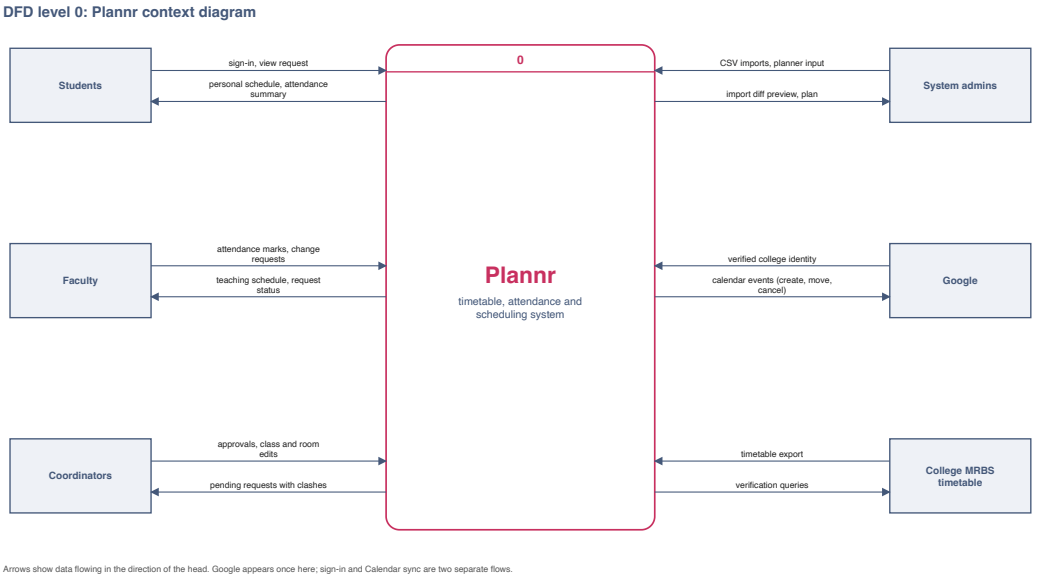


Figure 2: Data-flow diagram, level 0.

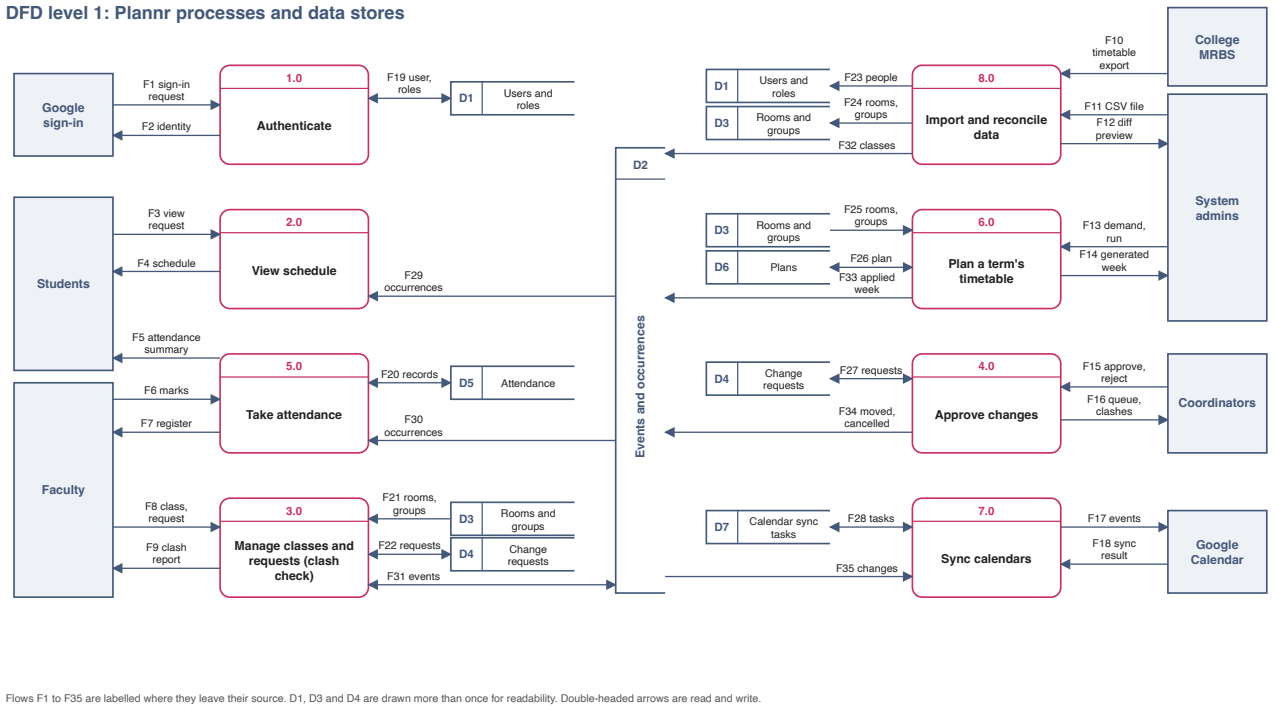


Figure 3: Data-flow diagram, level 1.

Key mechanisms

Recurrence. A class is stored once as an RFC 5545 recurrence rule (RRULE) and materialised into occurrence rows in the same transaction. Exceptions (a moved or cancelled single class) are stored separately, so editing a class once updates every timetable that shows it.

Approvals with advisory locks. Faculty request an extra lecture, a room change or a cancellation. Coordinators approve from one queue that shows clashes. Each approval takes a PostgreSQL advisory lock per room, teacher and group, inside the transaction, so two requests cannot both receive the same slot.

Attendance from the timetable. A register exists for every class because it is derived from the occurrence rows; there is no session to create. The teacher marks present, absent, late or excused, and students see attended, held and remaining per course.

The planner. Weekly demand per group is turned into a clash-free week: the most constrained sessions are placed first, then a repair step runs; unplaced sessions are listed with reasons. The plan goes through draft, in review, approved and applied, and is applied to the live timetable as a weekly pattern.

Calendar sync. Opt-in. Classes mirror into the person's Google Calendar through queued tasks drained by a cron job; moves and cancellations follow, and cancelled classes are retitled rather than deleted.

CSV import with preview. People and rooms are imported in bulk; the importer first computes a diff and shows every addition, change and removal, and applies it only after confirmation.

Legacy version. The `web-ui-compliance` branch implements the timetable viewer as PHP 8 (PDO) endpoints over a MySQL schema, with a jQuery and jQuery UI front end that fetches JSON by Ajax and builds the class cards in the DOM (details in Section 5.1).

Project files. The complete source code, schema and test suites are at https://github.com/paraswtf/plannr.

7 Screenshots

The frames below show Plannr's interface.

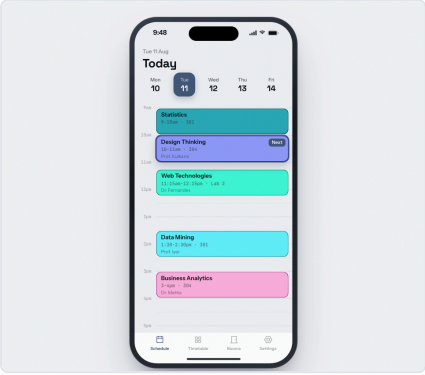


Figure 4: A student's day on a phone.

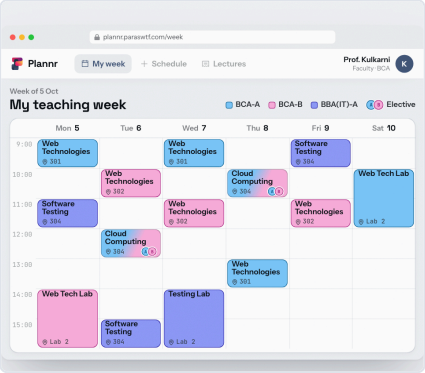


Figure 5: A teacher's week.

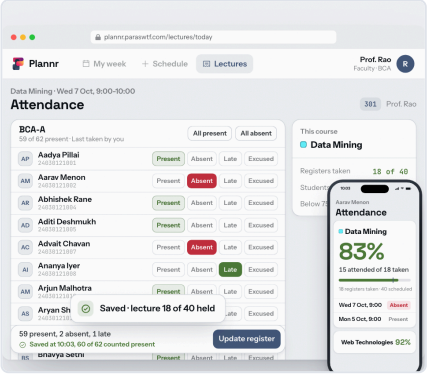


Figure 6: The attendance register and a student's course attendance.

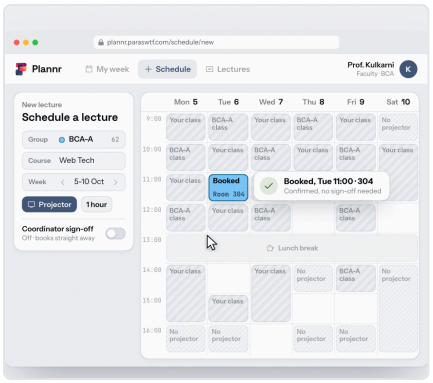


Figure 7: Scheduling an extra lecture into a clash-free slot.

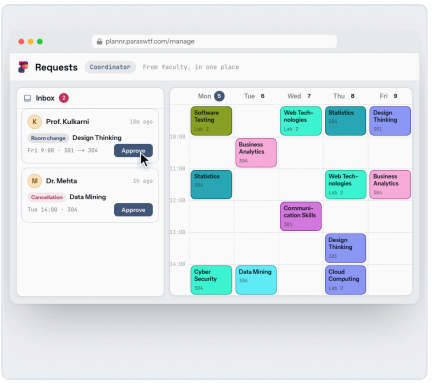


Figure 8: The coordinators' approvals queue.

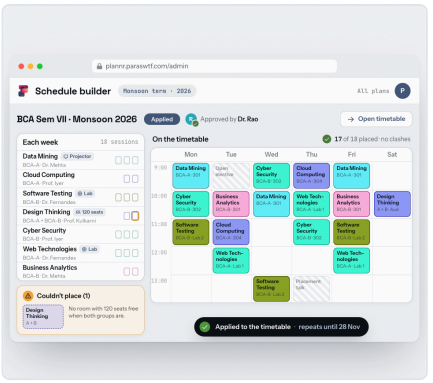


Figure 9: The timetable planner with the generated week applied.

8 Testing

Plannr has 49 Vitest test files in two groups: unit tests, and integration tests that run against a local PostgreSQL database.

Suite	What it checks
Unit: recurrence, dates	RRULE expansion and date handling
Unit: conflicts, free slots	Room, teacher, group and capacity clashes; next free slot
Unit: permissions	The permission bitfield and role defaults
Unit: CSV diff, timetable diff	Additions, changes and removals shown in previews
Unit: planner solver, schedule views	Generated weeks and the day, week and month views
Unit: pagination, OpenAPI, crypto	API paging, the OpenAPI document, key handling
Integration: timetable, events, courses, programmes	Database behaviour of the core entities
Integration: approvals	Request and approval flow, including the locks
Integration: attendance	Registers derived from the timetable
Integration: REST handler, personal API keys	The public API and scoped keys
Integration: Google Calendar sync	Mirroring, moves and cancellations

In addition, the script `db:verify-schedule` compares the database with the live college timetable class by class, which verifies the import from the college's LAMP system against its source.

9 Individual Contribution

Each member researched their stakeholders and developed the features for them.

Member	Contribution
Paras Deshpande	User research with system administrators; their requirements; the architecture, database and deployment; and development of the administrators' features: bulk CSV import, the timetable planner and the MRBS import and sync.
Sonal Mohrir	User research with students; their requirements; and development of the students' features: the personal day, week and month timetable, the attendance summary and Google Calendar sync.
Tanisha Nath	User research with faculty and coordinators; their requirements; and development of their features: the attendance register, requests for extra lectures, room changes and cancellations, the approvals queue, clash detection and room booking.

10 Conclusion

Plannr replaces a single desktop page with a personal, mobile-friendly timetable and adds attendance, clash-free scheduling and approvals that the research showed were done by hand. The project applies Ajax, the DOM and server-side database programming in a deployed system and connects to the college's existing LAMP timetable through a verified import.

11 Future Scope

The following are not built yet and are planned: push, email and in-app change notifications; the daily report of classes held; faculty availability entry; QR or self check-in; tests and marks; assignments and deadlines; file uploads and course material; an admin dashboard and audit log; and offline mode.

12 References

1. Next.js documentation. https://nextjs.org/docs
2. React documentation. https://react.dev
3. tRPC documentation. https://trpc.io/docs
4. Prisma documentation. https://www.prisma.io/docs
5. PostgreSQL documentation. https://www.postgresql.org/docs/
6. Supabase documentation. https://supabase.com/docs
7. Vercel documentation. https://vercel.com/docs
8. Auth.js documentation. https://authjs.dev
9. Tailwind CSS documentation. https://tailwindcss.com/docs
10. Google Calendar API. https://developers.google.com/calendar/api
11. RFC 5545: Internet Calendaring and Scheduling Core Object Specification (iCalendar). https://www.rfc-editor.org/rfc/rfc5545
12. MRBS (Meeting Room Booking System) documentation. https://mrbs.sourceforge.io/